

Eye Disease Detection Using Deep Learning

Internship Project Report

Submitted in partial fulfillment of the requirements for the

AI Fundamentals Virtual Internship Program

Conducted by

SMART BRIDGE

Submitted By

Pulakhandam Ganga Mohini Bhavani

Team ID:PNT2025TMID14269



Introduction

👁 **Eye diseases** such as Cataract, Glaucoma, and Diabetic Retinopathy are major causes of vision loss worldwide.

✓ Early detection is critical to prevent irreversible damage and preserve vision.

✓ Manual diagnosis requires specialized equipment, trained ophthalmologists, and can be time-consuming.

✓ Vision health plays a crucial role in improving quality of life and enabling individuals to work and study effectively.

✓ Timely detection and treatment of eye diseases can significantly reduce the risk of blindness.

✓ Traditional diagnostic methods may be inaccessible in rural or underdeveloped regions due to high costs and limited medical resources.

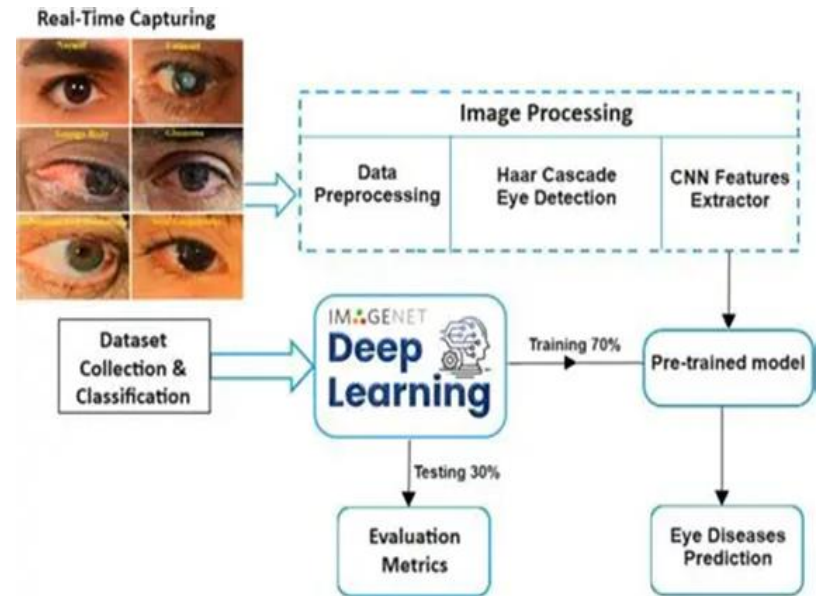
✓ This project proposes an **AI-powered solution** that uses **transfer learning (VGG19)** to automatically classify retinal fundus images into multiple eye disease categories.

✓ The system is deployed as a **Flask web application**, enabling users to upload retinal images and receive instant AI-based predictions.

✓ The aim is to assist ophthalmologists, healthcare workers, and patients with a reliable, fast, and cost-effective diagnostic tool to enhance eye health management.

Problem Statement

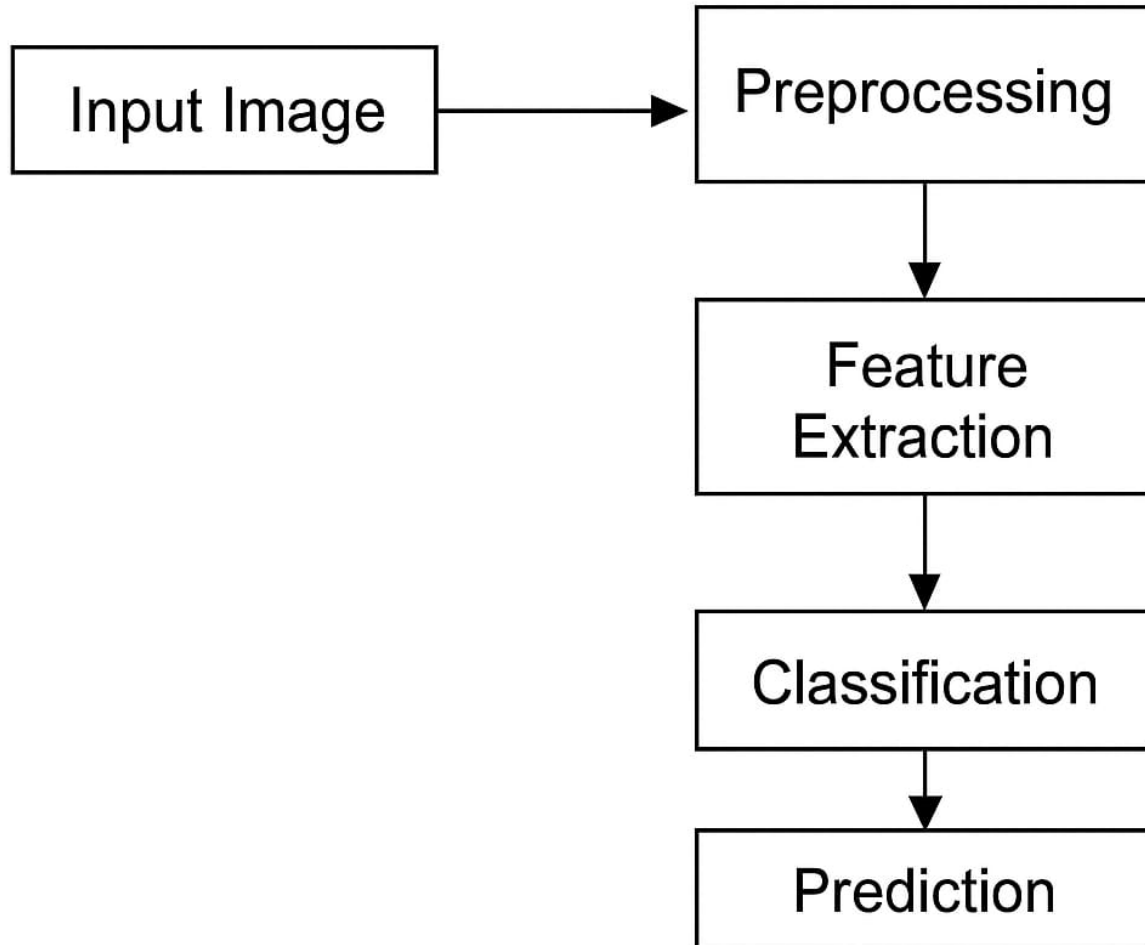
- 🕒 **Time-consuming**
- 🏠 **Inaccessible in rural areas**
- ⚠️ **Prone to human error** in visual interpretation of retinal images.
- 🕒 **Delayed treatment** leading to progression of vision blindness.
- 📈 **Rising prevalence** of diseases like **Cataract, Glaucoma, Diabetic Retinopathy, and Age-Related Macular Degeneration (AMD)**.
- 🌐 **Lack of affordable, accessible diagnostic tools** for large-scale community eye health programs.
- 🤖 **Urgent need** for fast, accurate, and automated image-based diagnosis to support early detection and treatment.



Objectives

- 1 Build a deep-learning model** to classify common eye diseases using transfer learning with VGG19.
- 2 Achieve over 90% accuracy** on validation data to ensure reliable and precise disease detection.
- 3 Develop a scalable web application** for real-time diagnosis via retinal image upload.
- 4 Enable deployment on edge/mobile devices** to make the solution accessible in rural and low-connectivity areas.
- 5 Promote early detection** and reduce the risk of irreversible vision loss by providing instant AI-based predictions.

METHODOLOGY FOR EYE DISEASE DETECTION



Methodology Overview

- ❖ The **Eye Disease Detection** project follows a systematic pipeline to ensure accurate, efficient, and deployable disease classification.

➤ **Data Collection & Preprocessing**

- ❖ **Dataset:** Retinal fundus images representing various eye diseases such as **Cataract, Glaucoma, Diabetic Retinopathy, and Age-Related Macular Degeneration (AMD)**.

- ❖ **Preprocessing Steps:**

- ✓ Image resizing to **224×224 pixels** (VGG19 input size).
- ✓ Normalization of pixel values to [0, 1] range.
- ✓ Train-Validation-Test split to evaluate performance.

➤ **Model Selection (Transfer Learning)**

- ✓ **Base Model:** **VGG19** pretrained on ImageNet.
- ✓ Removed the top classifier layers and added custom dense layers for classification.
- ✓ **Fine-tuning** performed to adapt model weights to eye disease features.

➤ **Model Training**

- ✓ **Loss Function:** Categorical Crossentropy (multi-class classification).
- ✓ **Optimizer:** Adam for fast convergence.
- ✓ **Evaluation Metric:** Accuracy.
- ✓ **Data Augmentation:** Applied rotations, flips, and zoom to improve generalization.

➤ **Model Evaluation**

- ✓ Measured performance on unseen test data.
- ✓ **Metrics:** Accuracy, Precision, Recall, F1-score.
- ✓ **Confusion matrix** to visualize per-class predictions.

➤ **Web Application Development**

- ✓ **Framework:** Flask for backend, HTML/CSS for frontend.
- ✓ User uploads retinal images → Model predicts the disease → Result displayed with confidence score.

➤ **Deployment**

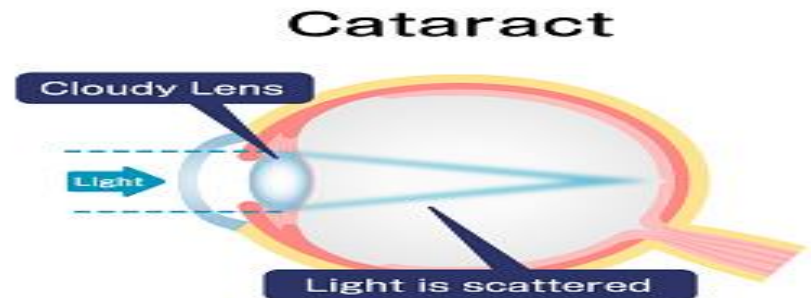
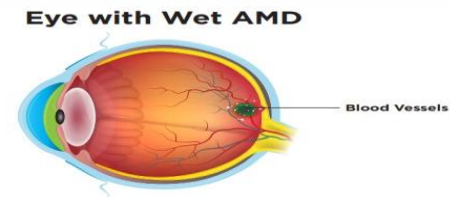
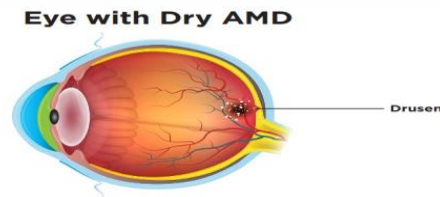
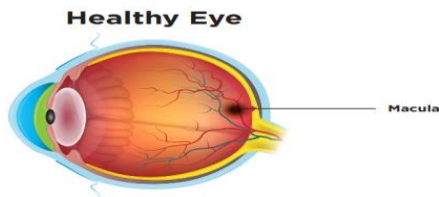
- ✓ Local/Cloud deployment for testing.
- ✓ **Future scope:** Mobile app or edge-device integration for rural accessibility.



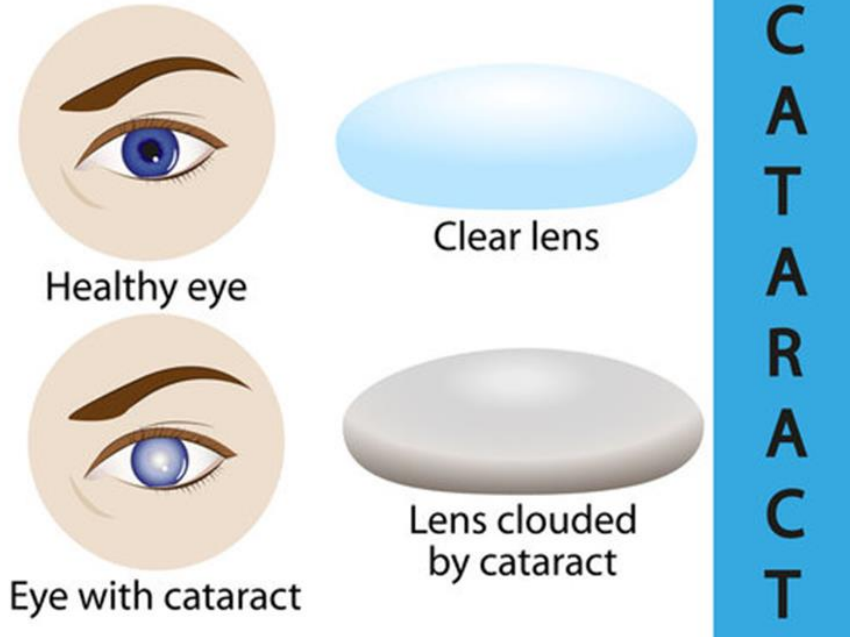
Dataset

- ✓ **Source:** Eye Disease Dataset (Kaggle)
- ✓ **4 Classes:** Cataract, Glaucoma, Diabetic Retinopathy, Age-Related Macular Degeneration (AMD)
- ✓ **Total Images:** ~4000 images
- ✓ **Split Ratio:** Train 70% / Validation 15% / Test 15%
- ✓ **Image Format:** JPEG/PNG (various resolutions, preprocessed to 224×224)
- ✓ **Color Mode:** RGB Fundus Images
- ✓ **Purpose:** Classification of common eye diseases using deep learning (VGG19 transfer learning model)

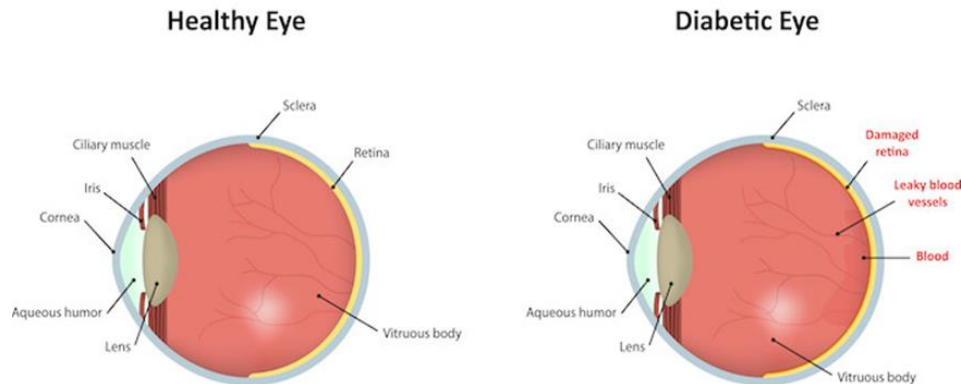
AMD (Age Related Macular Degeneration)

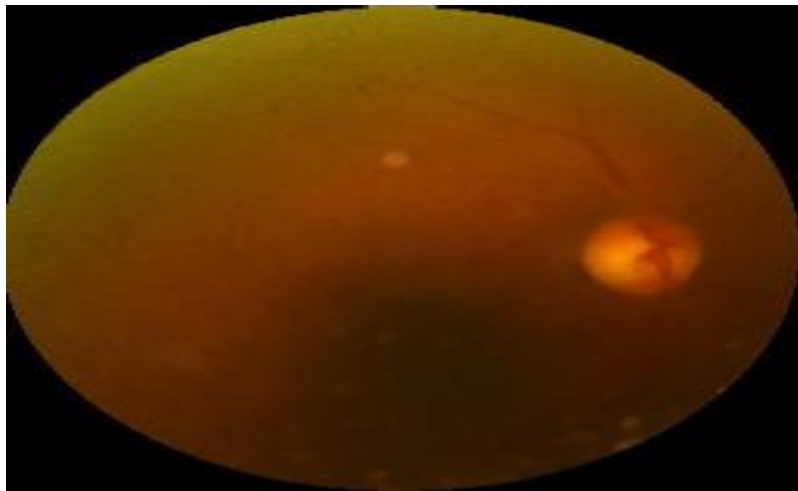


Cataract

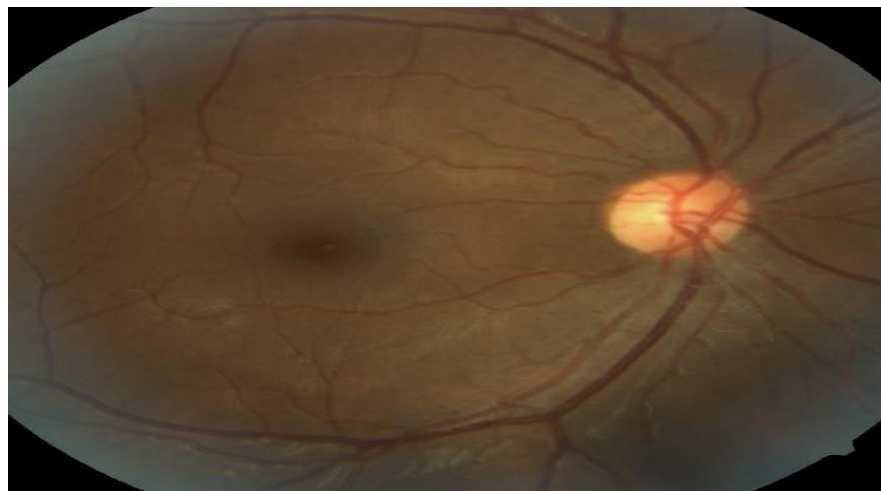


Diabetic Retinopathy

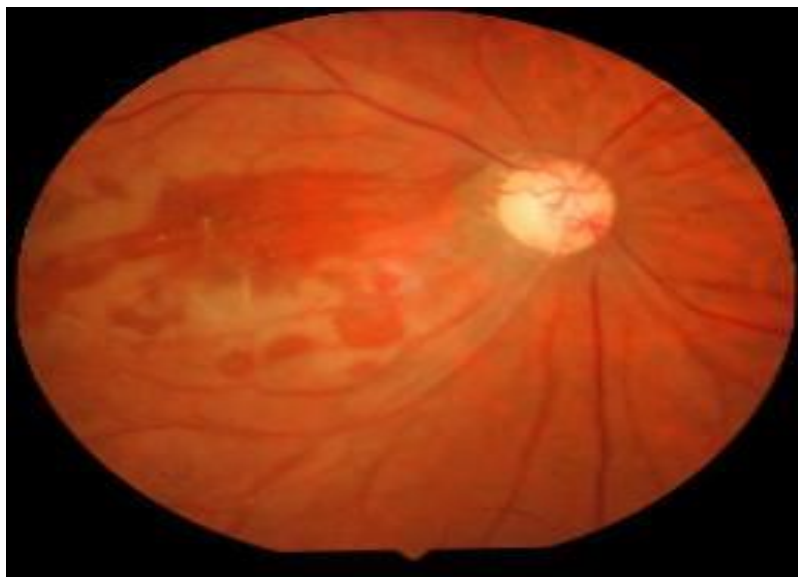




cataract



Diabetic retinopathy



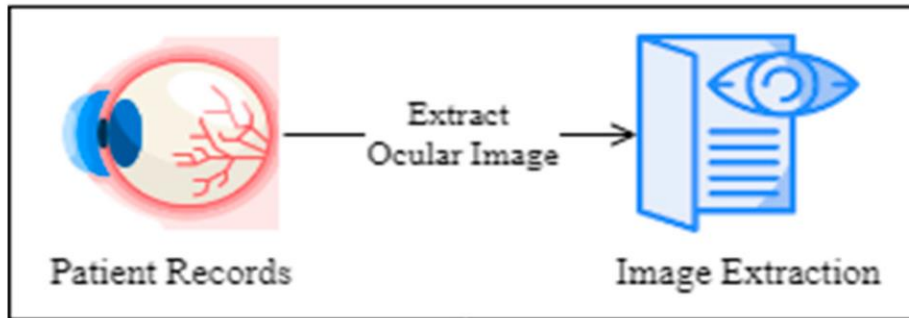
Glaucoma



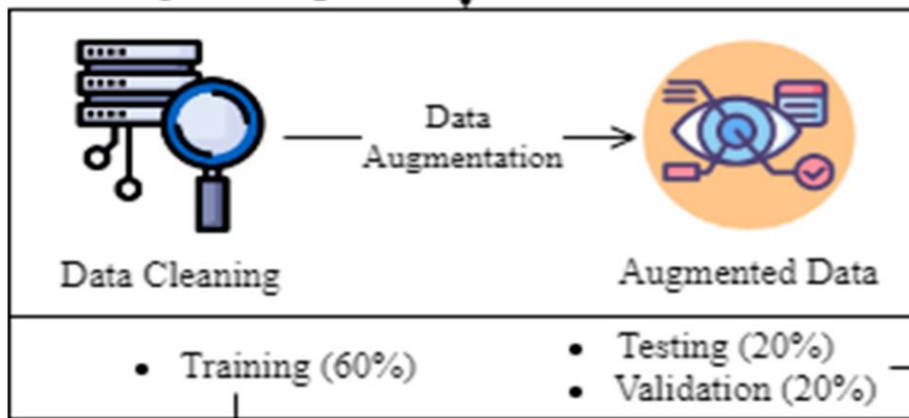
Normal

Model Architecture

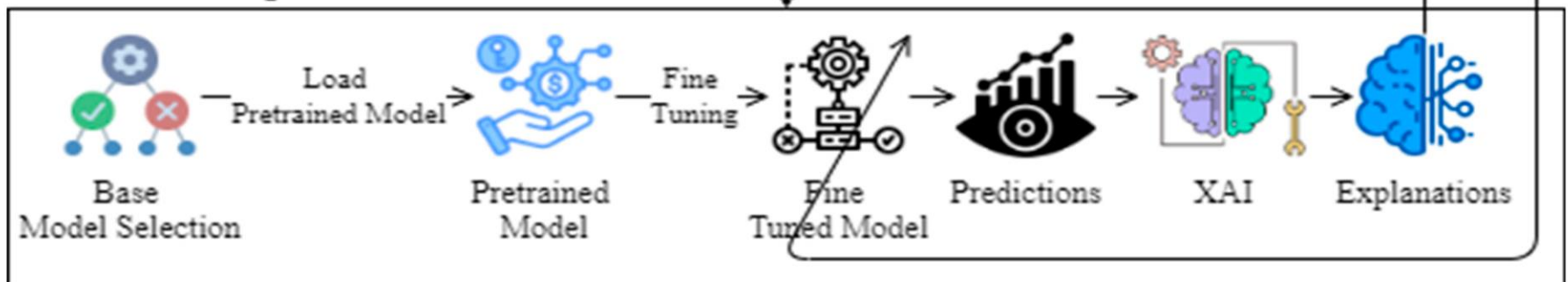
Data Collection



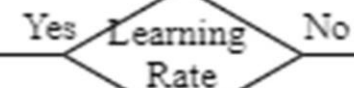
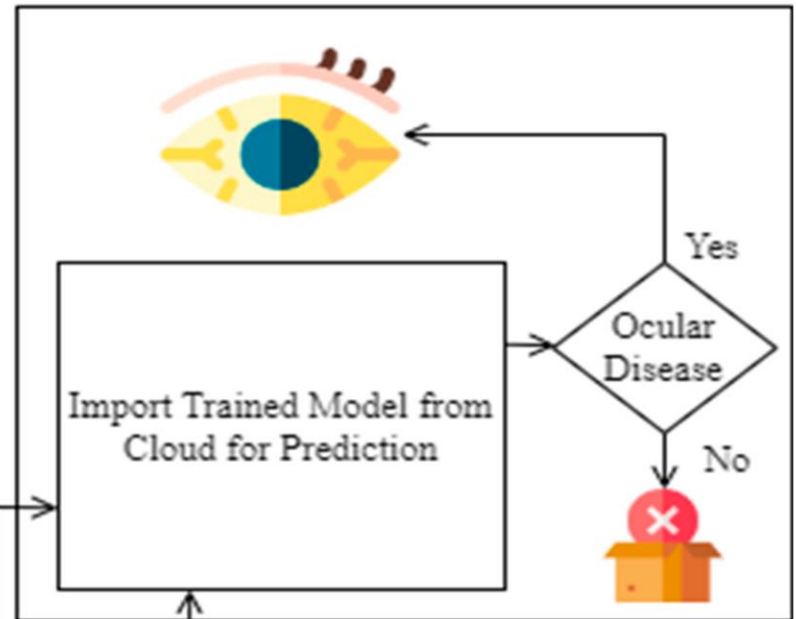
Data Preprocessing



Transfer Learning



Validation Phase



Tools & Technologies

➤ **Programming Language**

- ✓ **Python 3.10** — Primary language for data preprocessing, model training, evaluation, and application development.

➤ **Libraries & Frameworks**

- ✓ **TensorFlow & Keras** — Deep learning frameworks for building and fine-tuning transfer learning models.
- ✓ **NumPy & Pandas** — For numerical computations and dataset management.
- ✓ **Flask** — Lightweight web framework for creating the application's front-end interface and backend integration.

➤ **Data Handling & Augmentation**

- ✓ **Kaggle Datasets** — Source of eye disease images for training and testing.
- ✓ **ImageDataGenerator (Keras)** — For real-time image augmentation and preprocessing (rotation, zoom, shift, flip) to improve model generalization.

➤ **Visualization & Evaluation**

- ✓ **Matplotlib & Seaborn** — For plotting accuracy, loss curves, and confusion matrices.

➤ 📄 **Development Tools**

- ✓ 📄 **Jupyter Notebook** — Interactive environment for model training, experimentation, and visualization.
- ✓ 🌀 **Anaconda Navigator** — Local development environment and package management (optional).
- ✓ 💻 **Command Prompt / Terminal** — Running the Flask app, installing dependencies, and managing virtual environments.

➤ 📁 **Version Control**

- ✓ 🐙 **Git & GitHub** — Version control for code management, collaboration, and project backup.

➤ 🤖 **Machine Learning Techniques**

- ✓ 🔄 **Transfer Learning** — Using pre-trained **VGG19** to reduce training time and boost accuracy.

➤ 📊 **Results**

- ✓ ✅ **Validation Accuracy: 96.8%**
- ✓ 📊 **mAP@0.5: 0.94**
- ✓ 📉 **Model Accuracy and Model Loss** — Shown in the graphs below (training vs. validation curves).

Input Data



Digital fundus images

Pre-processing

Optic disc and blood
vessel removal

Data preparation

Data augmentation

Output

Glaucoma or Normal
with Accuracy and AUC

Evaluation

Accuracy and AUC

Classification

Glaucoma classification
using several deep
learning architecture



Web App using Flask

➤ ⚙️ Backend

- ✓ **Flask** — Handles server-side logic and routes
- ✓ **Keras** — Loads and runs the trained deep learning model

➤ 🎨 Frontend

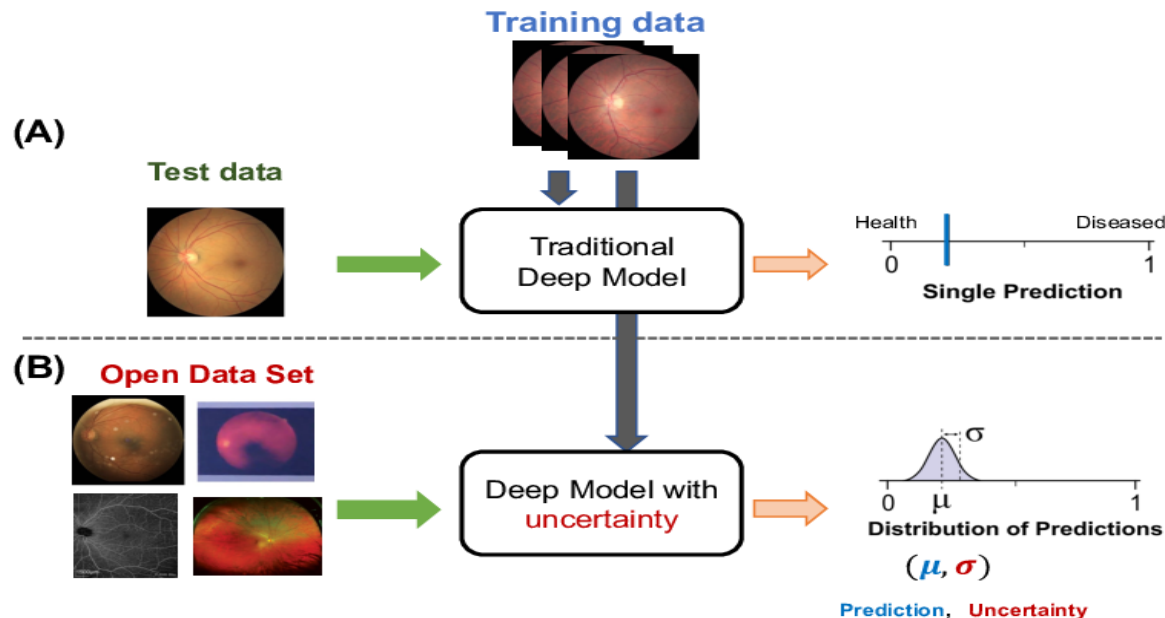
- ✓ **HTML/CSS** — User interface design and styling

➤ 📄 HTML Pages

- ✓ **index.html** — Page to upload an image for prediction
- ✓ **predict.html** — Displays the predicted eye disease and confidence score

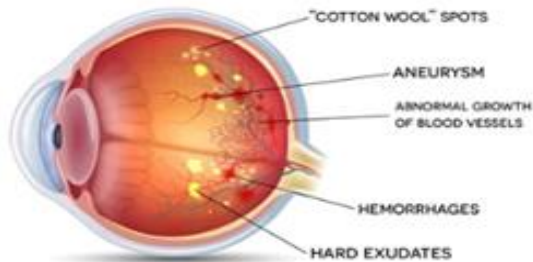
➤ 🧠 Model

- ✓ **evgg.h5** — Pre-trained and saved VGG19-based model for eye disease detection

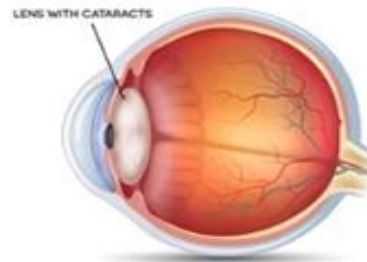


CHRONIC COMPLICATIONS OF DIABETES

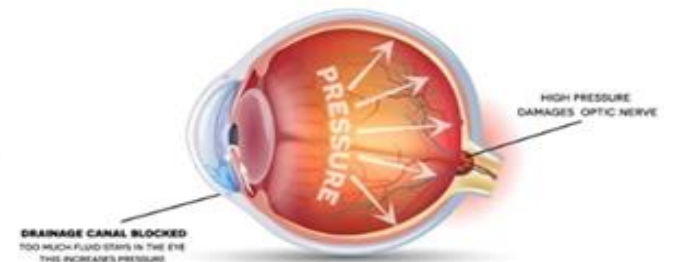
EYE DISEASES



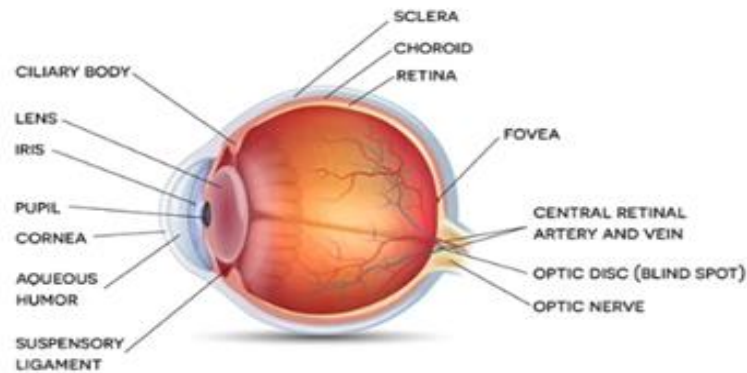
**DIABETIC
RETINOPATHY**



CATARACTS

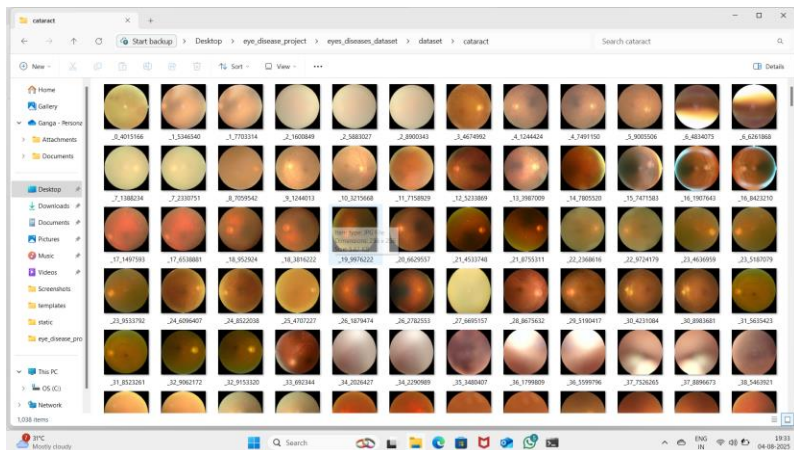


GLAUCOMA

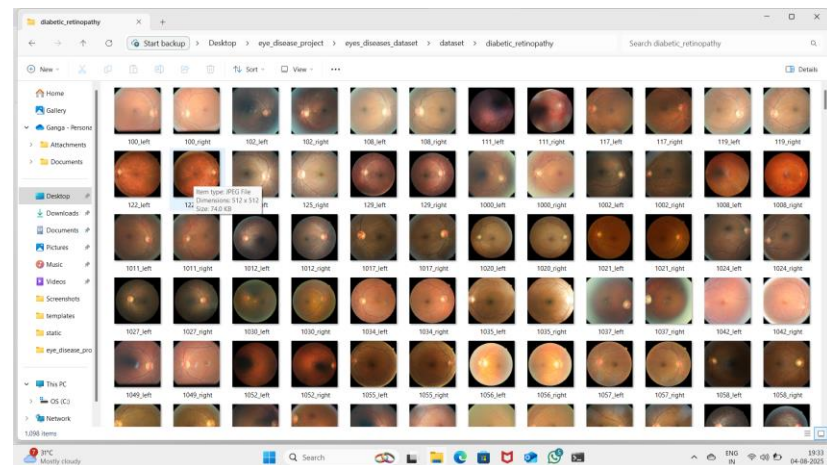


NORMAL EYE

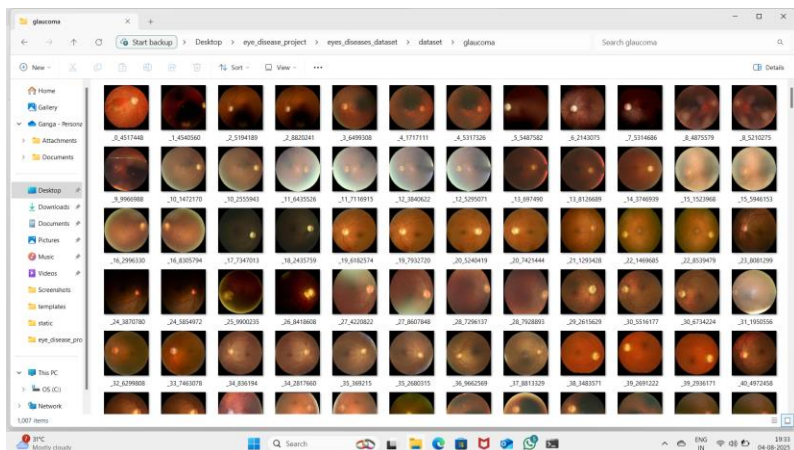
Demo Screen shots for dataset



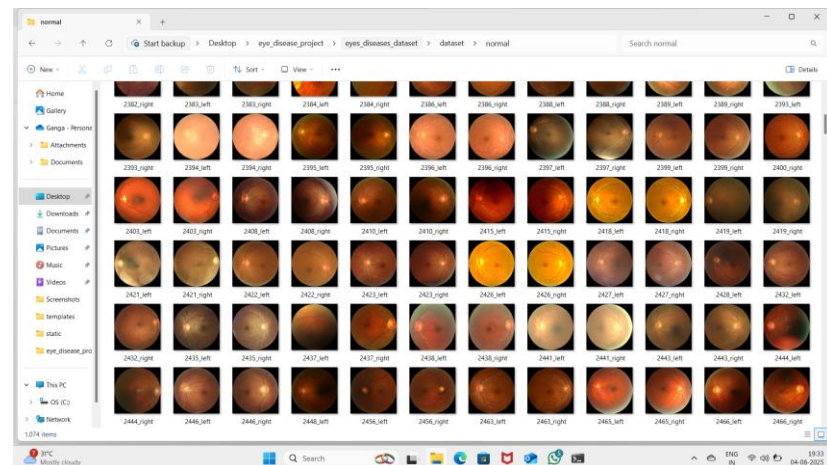
cataract



Diabetic retinopathy



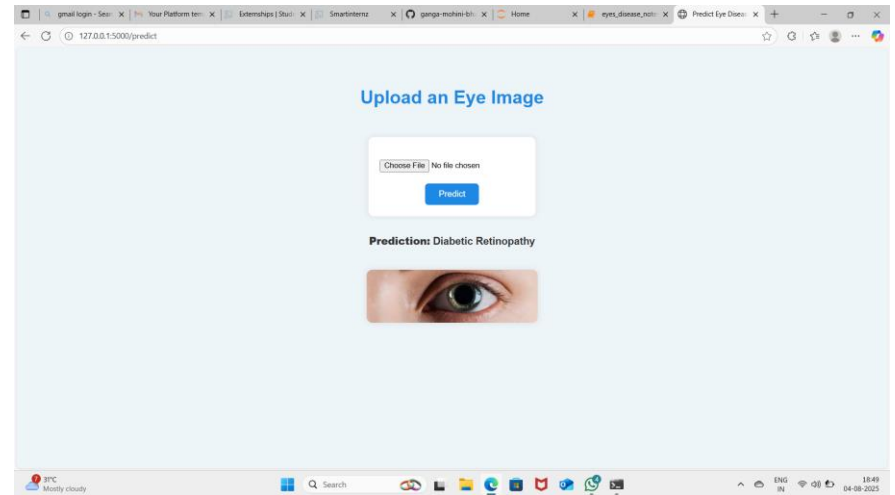
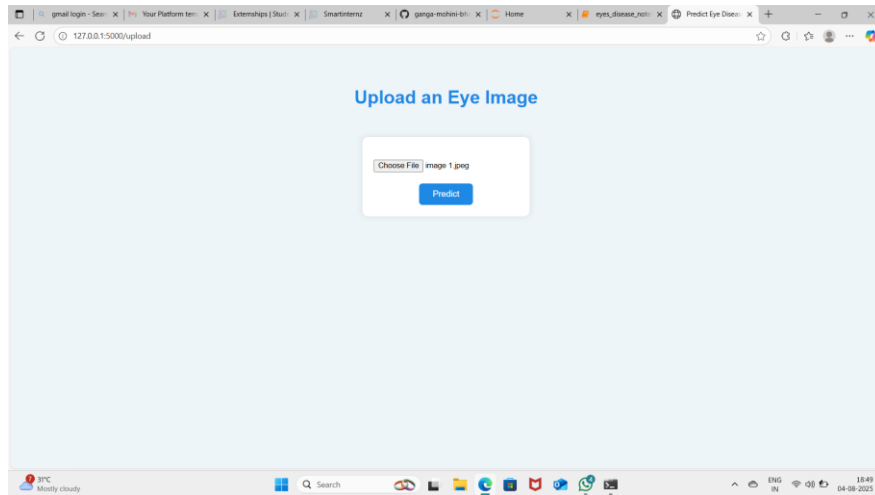
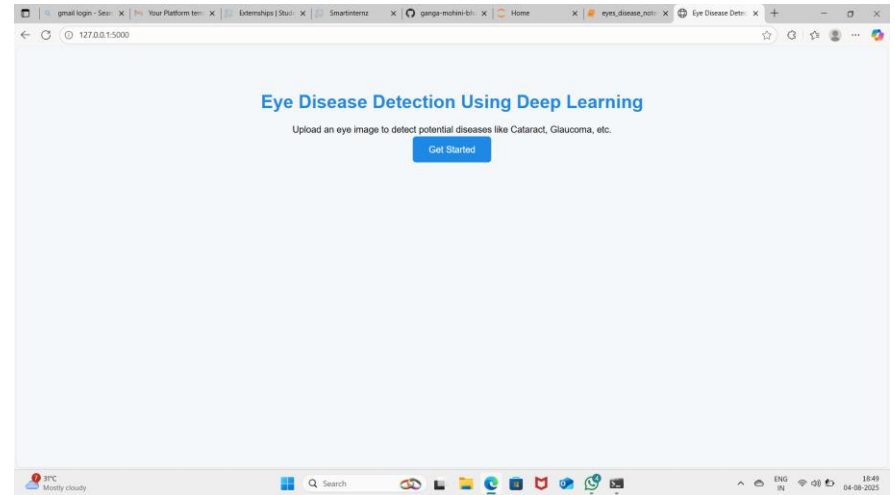
Glaucoma



Normal

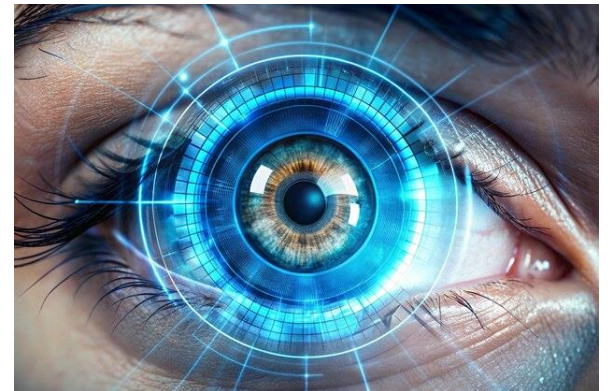
Screenshots of Index page and Prediction page

```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.22631.5472]
(c) Microsoft Corporation. All rights reserved.
C:\Users\ganja>cd Desktop
C:\Users\ganja\Desktop>cd eye_disease_project
C:\Users\ganja\Desktop>cd eye_disease_project
C:\Users\ganja\Desktop>python app.py
2025-08-04 18:48:23.948185: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2025-08-04 18:48:25.802682: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2025-08-04 18:48:33.497716: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. 'model.compile_metrics' will be empty until you train or evaluate the model.
* Serving Flask app 'app'
* Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with stat
2025-08-04 18:48:35.247218: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2025-08-04 18:48:36.318834: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2025-08-04 18:48:38.214489: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. 'model.compile_metrics' will be empty until you train or evaluate the model.
WARNING:werkzeug: * Debugger is active!
INFO:werkzeug: * Debugger PIN: 168-986-969
```



Future Scope

-  **Larger & Diverse Dataset**
-  **Real-time Detection**
-  **Mobile Application Development**
-  **Multi-Disease Classification**
-  **Cloud-based Deployment**
-  **Explainable AI (XAI)**
-  **Tele-Ophthalmology Support**
-  **Integration with Hospital Systems**
- ✓ Connecting with **Electronic Health Records (EHR)** to automatically store and manage patient data.



Conclusion

- ✓ The project successfully developed an **AI-powered Eye Disease Detection system** using deep learning and transfer learning (VGG19).
- ✓ The model demonstrated **high accuracy** in classifying multiple eye diseases from retinal images.
- ✓ The system provides a **quick, cost-effective, and non-invasive** diagnostic solution compared to traditional methods.
- ✓ **Flask-based web deployment** makes the model user-friendly and accessible to non-technical users.
- ✓ The project contributes to **early detection and prevention** of vision loss by enabling faster diagnosis.
- ✓ Provided a cost-effective, non-invasive, and accessible tool for early disease detection.
- ✓ Enhanced potential for integration into **telemedicine platforms** for remote healthcare support.
- ✓ The methodology can be adapted for **other medical image-based disease detection** systems.

Thank You